

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

PEDRO F. LEÃO

LUCAS P. FLORES

**DESENVOLVIMENTO DE UM SISTEMA DE *HIGH-FREQUENCY TRADING* DE
BAIXO CUSTO UTILIZANDO HLS EM FPGA**

CURITIBA

2026

**PEDRO F. LEÃO
LUCAS P. FLORES**

**DESENVOLVIMENTO DE UM SISTEMA DE *HIGH-FREQUENCY TRADING* DE
BAIXO CUSTO UTILIZANDO HLS EM FPGA**

**Development of a Low-Cost High-Frequency Trading System using HLS on
FPGA**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção
do título de Bacharel em Engenharia de
Computação do Curso de Engenharia de
Computação da Universidade Tecnológica
Federal do Paraná.

Orientador: Prof. Carlos Raimundo Erig Lima

**CURITIBA
2026**



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

**PEDRO F. LEÃO
LUCAS P. FLORES**

**DESENVOLVIMENTO DE UM SISTEMA DE *HIGH-FREQUENCY TRADING* DE
BAIXO CUSTO UTILIZANDO HLS EM FPGA**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção
do título de Bacharel em Engenharia de
Computação do Curso de Engenharia de
Computação da Universidade Tecnológica
Federal do Paraná.

Data de aprovação:

Prof. Carlos Raimundo Erig Lima
Orientador
Universidade Tecnológica Federal do Paraná, Campus Curitiba

Prof. Daniel Fernando Pigatto
Membro da banca
Universidade Tecnológica Federal do Paraná, Campus Curitiba

Prof. Elder Oroski
Membro da banca
Universidade Tecnológica Federal do Paraná, Campus Curitiba

**CURITIBA
2026**

Aos familiares, professores e colegas que
sustentaram a trajetória acadêmica que tornou
este trabalho possível.

Agradecimentos

Agradecemos ao Prof. Carlos Raimundo Erig Lima pela orientação, pelas discussões técnicas e pela confiança no desenvolvimento de um tema que combina arquitetura de computadores, sistemas embarcados e aplicações financeiras de baixa latência.

Agradecemos à Universidade Tecnológica Federal do Paraná, Campus Curitiba, pela formação acadêmica e pelo ambiente de aprendizagem que permitiu integrar conhecimentos de programação, eletrônica digital, redes de computadores, síntese de hardware e metodologia científica.

Agradecemos também aos colegas, familiares e amigos que contribuíram com apoio, revisão, testes, questionamentos e incentivo durante a elaboração deste trabalho.

Resumo

Este trabalho desenvolve e avalia um protótipo acadêmico que integra um processador ARM e um arranjo de portas programáveis em campo (FPGA) para processar dados sintéticos de mercado e executar uma regra de decisão gerada por síntese de alto nível. Um gerador transmite pelo Protocolo de Controle de Transmissão (TCP) mensagens sintéticas codificadas segundo *FIX Adapted for STreaming* (FAST), e um receptor C++ as decodifica e converte em quadros de 256 bits. No FPGA da placa DE10-Nano, filas circulares transportam os eventos, um livro de ofertas agregado mantém oito níveis por lado e uma função escrita em MATLAB, convertida para VHDL pelo HDL Coder, produz os códigos NOOP, BUY ou SELL. A ponte, o livro e os invólucros permanecem em VHDL manual. A validação compreende testes C++, bancos de teste VHDL e um ensaio funcional na placa. Em uma campanha registrada com um milhão de eventos, o núcleo FPGA apresentou latência de três ciclos (60 ns a 50 MHz), com variação desprezível na resolução de 20 ns do contador, enquanto o núcleo C++ de referência apresentou média de 315 ns. A latência média de ida e volta pela entrada e saída mapeada em memória foi 10.228 ns, com percentil 99 de 10.310 ns, e a vazão em fluxo foi 102.381 mensagens por segundo. Os valores caracterizam uma execução específica e não incluem TCP ou decodificação FAST. O protótipo não se conecta a bolsa, não envia ordens, não implementa controle de risco e não constitui um sistema operacional de negociação de alta frequência.

Palavras-chave: fpga; *high-frequency trading*; síntese de alto nível; livro de ofertas; sistemas embarcados.

Abstract

This work develops and evaluates an academic prototype that integrates an ARM processor and a field-programmable gate array (FPGA) to process synthetic market data and execute a decision rule generated through high-level synthesis. A generator transmits synthetic FIX Adapted for STreaming (FAST) messages over the Transmission Control Protocol (TCP), and a C++ receiver decodes them and converts their fields into 256-bit frames. In the DE10-Nano FPGA, circular queues transport the events, an aggregated order book maintains eight levels per side, and a MATLAB function converted to VHDL by HDL Coder produces NOOP, BUY, or SELL codes. The bridge, order book, and wrappers remain manually written in VHDL. Validation comprises C++ tests, VHDL testbenches, and a functional run on the board. In one recorded campaign with one million events, the FPGA core showed a three-cycle latency (60 ns at 50 MHz), with negligible variation at the counter resolution of 20 ns, whereas the reference C++ core averaged 315 ns. The average round-trip latency through the memory-mapped input/output path was 10,228 ns, its 99th percentile was 10,310 ns, and the streaming throughput was 102,381 messages per second. These values characterize one specific run and exclude TCP or FAST decoding. The prototype does not connect to an exchange, send orders, implement risk controls, or constitute an operational high-frequency trading system.

Keywords: fpga; high-frequency trading; high-level synthesis; order book; embedded systems.

Declaração de Uso de Inteligência Artificial Generativa

Durante a elaboração deste trabalho, foram utilizadas as seguintes ferramentas de Inteligência Artificial Generativa:

- **Claude Sonnet 4.6** (Anthropic): utilizado para auxílio na revisão e escrita do texto acadêmico, geração e refatoração de código (C++, VHDL, MATLAB e rotinas de automação de compilação), e geração de casos de teste para os módulos de software e hardware.
- **GPT-5.4** (OpenAI): utilizado para auxílio na revisão e escrita do texto acadêmico, geração e refatoração de código, e geração de casos de teste.

Os autores declaram que o conteúdo produzido com auxílio das ferramentas acima — incluindo texto, código-fonte e casos de teste — foi revisado por eles, que assumem responsabilidade pela precisão, integridade ética e originalidade do resultado final.

Lista de Figuras

Figura 1 – Placa DE10-Nano utilizada no protótipo	15
Figura 2 – Arquitetura geral do protótipo	24
Figura 3 – Fluxo lógico do livro de ofertas simplificado	25
Figura 4 – Ligação entre o HPS e o periférico no Platform Designer	30
Figura 5 – Fluxo de um evento no ensaio funcional	31
Figura 6 – Procedimento de alteração e geração da estratégia	33

Lista de Tabelas

Tabela 1 – Quadro enviado do ARM para o FPGA	26
Tabela 2 – Quadro devolvido do FPGA para o ARM	26
Tabela 3 – Mapeamento de símbolos do fluxo sintético	27
Tabela 4 – Evidências do ensaio funcional na DE10-Nano	39
Tabela 5 – Valores registrados na campanha quantitativa	40
Tabela 6 – Estimativa de energia por decisão sob as hipóteses adotadas	42
Tabela 7 – Contextualização das latências reportadas	42
Tabela 8 – Registradores da interface MMIO	49
Tabela 9 – Formato do quadro ARM–FPGA	50
Tabela 10 – Variáveis de ambiente do receptor	55

Lista de Quadros

Quadro 1 – Relação entre a literatura e o escopo do protótipo	22
Quadro 2 – Materiais empregados no desenvolvimento	29
Quadro 3 – Procedimentos de validação funcional	34
Quadro 4 – Métricas e respectivos escopos de medição	37
Quadro 5 – Resultado da execução dos testes automatizados	39

Listagem de Códigos Fonte

Listagem 1 – Pseudocódigo da regra de decisão	27
Listagem 2 – Trecho da saída do receptor com respostas FPGA–ARM	38
Listagem 3 – Comandos de teste local	53
Listagem 4 – Geração do bloco de estratégia por HDL Coder	53
Listagem 5 – Fluxo previsto para DE10-Nano	54
Listagem 6 – Execução na placa	54

Lista de Abreviaturas e Siglas

Siglas

ABNT	Associação Brasileira de Normas Técnicas
API	Interface de Programação de Aplicações, do inglês <i>Application Programming Interface</i>
ARM	Família de arquiteturas de processadores RISC utilizada no subsistema embarcado
CPU	Unidade Central de Processamento, do inglês <i>Central Processing Unit</i>
DMA	Acesso Direto à Memória, do inglês <i>Direct Memory Access</i>
FAST	FIX Adapted for STreaming
FIX	Financial Information eXchange
FPGA	Arranjo de Portas Programáveis em Campo, do inglês <i>Field-Programmable Gate Array</i>
GHDL	Simulador de VHDL utilizado nos testes automatizados
HDL	Linguagem de Descrição de Hardware, do inglês <i>Hardware Description Language</i>
HFT	Negociação de Alta Frequência, do inglês <i>High-Frequency Trading</i>
HLS	Síntese de Alto Nível, do inglês <i>High-Level Synthesis</i>
HPS	Sistema Processador Rígido, do inglês <i>Hard Processor System</i>
IP	Propriedade Intelectual reutilizável de hardware, do inglês <i>Intellectual Property</i>
MMIO	Entrada e Saída Mapeada em Memória, do inglês <i>Memory-Mapped Input/Output</i>
PDF	Formato de Documento Portátil, do inglês <i>Portable Document Format</i>
RTL	Nível de Transferência entre Registradores, do inglês <i>Register Transfer Level</i>
SoC	Sistema em um Chip, do inglês <i>System-on-Chip</i>
TCP	Protocolo de Controle de Transmissão, do inglês <i>Transmission Control Protocol</i>
UTFPR	Universidade Tecnológica Federal do Paraná
VCD	Value Change Dump
VHDL	VHSIC Hardware Description Language

Sumário

1	Introdução	14
1.1	Problema de pesquisa	15
1.2	Objetivos	15
1.2.1	Objetivo geral	15
1.2.2	Objetivos específicos	15
1.3	Justificativa	16
1.4	Estrutura do trabalho	16
2	Referencial teórico	18
2.1	Negociação algorítmica e HFT	18
2.2	Livro de ofertas e dados de mercado	18
2.3	FPGA, SoC e HPS	19
2.4	Síntese de alto nível e MATLAB HDL Coder	19
2.5	FAST e mFAST	20
2.6	MMIO, Avalon-MM e filas circulares	20
3	Trabalhos relacionados	21
3.1	Negociação algorítmica e baixa latência	21
3.2	Aceleração em FPGA	21
3.3	Livro de ofertas	21
3.4	Posicionamento do protótipo	22
4	Proposta	23
4.1	Arquitetura e fronteiras	23
4.2	Decisões arquiteturais	24
4.3	Contrato ARM–FPGA	26
4.4	Símbolos, regra de decisão e delimitação	27
5	Materiais e métodos	29
5.1	Materiais	29
5.2	Procedimento de implementação	29
5.2.1	Integração no Platform Designer	30
5.2.2	Fluxo sintético de dados	31

5.2.3	Geração e integração da estratégia	31
5.3	Validação funcional	34
5.4	Método da avaliação quantitativa	34
5.5	Método do experimento de cálculo energético	37
6	Resultados e discussão	38
6.1	Execução funcional na DE10-Nano	38
6.2	Resultados dos testes automatizados	39
6.3	Campanha quantitativa	39
6.3.1	Variação interna e resolução do contador	41
6.3.2	Comunicação e vazão	41
6.3.3	Resultado do experimento de cálculo energético	42
6.4	Discussão frente aos trabalhos relacionados	42
6.4.1	Limitações da evidência	43
7	Conclusão	44
	REFERÊNCIAS	46
	APÊNDICE A Contrato de Comunicação ARM–FPGA	49
	A.1 Registradores	49
	A.2 Regras de filas	49
	A.3 Formato de evento	50
	A.4 Formato de resposta	50
	A.5 Checklist de alteração do contrato	51
	APÊNDICE B Comandos de Construção, Teste e Execução	53
	B.1 Validação local	53
	B.2 Geração da estratégia	53
	B.3 Geração HDL e compilação para DE10-Nano	53
	B.4 Variáveis de ambiente	55
	B.5 Sequência recomendada de depuração	55

1 Introdução

Negociação algorítmica é o uso de programas para produzir decisões de negociação a partir de regras previamente definidas. Dentro desse campo, a negociação de alta frequência, do inglês *High-Frequency Trading* (HFT), caracteriza uma classe de sistemas automatizados em que a latência de processamento e comunicação influencia a capacidade de reagir a eventos de mercado (ALDRIDGE, 2013; HASBROUCK; SAAR, 2013). Uma dessas fontes de eventos é o livro de ofertas, denominado *order book* em inglês: a estrutura que organiza ofertas de compra e venda e permite identificar, entre outros dados, os melhores preços de cada lado (GOULD *et al.*, 2013).

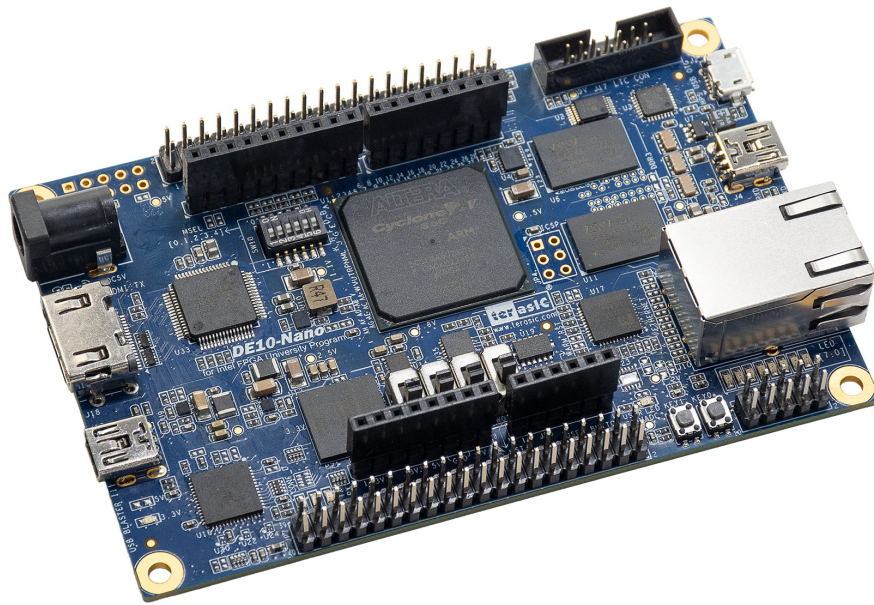
O processamento de baixa latência não depende apenas da regra de decisão. Ele compreende recepção e decodificação de mensagens, representação de preços e quantidades, atualização de estado e comunicação entre os componentes. Processadores de propósito geral oferecem flexibilidade para essas tarefas, mas o tempo observado por uma aplicação também depende da hierarquia de memória, do sistema operacional e de outras atividades concorrentes. Um arranjo de portas programáveis em campo, do inglês *Field-Programmable Gate Array* (FPGA), permite implementar parte do fluxo como circuito dedicado e explorar paralelismo espacial; isso não torna automaticamente determinístico o sistema completo, sobretudo quando rede, processador e sistema operacional continuam no caminho de dados (PATTERSON; HENNESSY, 2017; HARRIS; HARRIS, 2013).

Este trabalho estuda esse recorte por meio de um protótipo acadêmico baseado na placa Intel/Terasic DE10-Nano, que reúne processadores ARM e lógica FPGA no mesmo sistema em chip (Terasic Technologies, 2024; Intel Corporation, 2025). A proposta mantém em software a geração e a decodificação do fluxo sintético de dados e implementa em hardware a comunicação por filas, um livro de ofertas simplificado e uma regra de decisão. A regra é descrita em MATLAB e convertida em VHDL, uma linguagem de descrição de hardware, pelo MATLAB HDL Coder, ferramenta de síntese de alto nível, do inglês *High-Level Synthesis* (HLS) (The MathWorks, Inc., 2026b). A HLS não gera, neste projeto, o receptor de rede, a ponte de comunicação nem o livro de ofertas.

A placa usada na integração é apresentada na Figura 1. Sua escolha decorre da presença de processadores, FPGA e pontes de comunicação no mesmo dispositivo (Terasic Technologies, 2024), característica suficiente para investigar a divisão entre software e hardware em ambiente acadêmico. Essa escolha, porém, não implica equivalência técnica com equipamentos empregados em negociação profissional.

Com esse recorte, o protótipo não se conecta a uma bolsa, não recebe um fluxo real de mercado, não envia ordens e não implementa controle de risco ou conformidade regulatória. O termo HFT funciona como contexto de aplicação do estudo, e não como classificação operacional do sistema construído. A validação abrange o fluxo sintético, a integração ARM-FPGA, a lógica implementada e uma campanha inicial de desempenho.

Figura 1 – Placa DE10-Nano utilizada no protótipo



Fonte: Terasic Technologies (Terasic Technologies, 2024).

1.1 Problema de pesquisa

A pergunta de pesquisa é: como estruturar, em uma plataforma ARM–FPGA acadêmica, um fluxo simplificado de dados de mercado no qual a regra de decisão possa ser gerada a partir de MATLAB sem alterar o contrato de comunicação e o livro de ofertas?

A pergunta concentra duas preocupações. A primeira é arquitetural, pois software e hardware precisam compartilhar formato de evento, mapa de registradores e regras de publicação das filas. A segunda é de desenvolvimento, uma vez que a estratégia deve ser substituível sem transferir para o HDL Coder a responsabilidade por toda a infraestrutura. O estudo avalia essa separação por testes funcionais e por métricas do núcleo lógico e do caminho ARM–FPGA–ARM.

1.2 Objetivos

1.2.1 Objetivo geral

Desenvolver e avaliar um protótipo acadêmico ARM–FPGA para processar eventos sintéticos de mercado, mantendo a regra de decisão como bloco MATLAB gerado para VHDL e as interfaces de comunicação e estado como infraestrutura previamente definida.

1.2.2 Objetivos específicos

Os objetivos específicos são:

- implementar um gerador e um receptor de mensagens sintéticas codificadas segundo o padrão *FIX Adapted for STreaming* (FAST) e transmitidas pelo Protocolo de Controle de Transmissão (TCP);
- definir e documentar o contrato binário entre o software ARM e a lógica FPGA;
- implementar, no FPGA, um livro por nível de preço, com profundidade fixa e sinais de melhor compra, melhor venda, diferença entre preços e desbalanceamento de quantidades;
- descrever uma regra de decisão sintetizável em MATLAB, gerar seu VHDL e integrá-la ao motor de processamento;
- verificar o software, as filas, o livro, a estratégia e o motor integrado por testes automatizados e por execução funcional na DE10-Nano;
- caracterizar, em uma campanha experimental, a latência interna do núcleo FPGA, a latência de ida e volta pela interface mapeada em memória, a vazão e o tempo de uma implementação C++ de referência.

1.3 Justificativa

O trabalho relaciona arquitetura de computadores, sistemas embarcados e processamento de dados financeiros. A literatura mostra que a baixa latência afeta a interação entre participantes de mercados eletrônicos, embora os efeitos econômicos dependam do mercado e da estratégia analisados (HENDERSHOTT; JONES; MENKVELD, 2011; BROGAARD; HENDERSHOTT; RIORDAN, 2014; HASBROUCK; SAAR, 2013). Este estudo não avalia esses efeitos econômicos; o domínio financeiro é usado como caso para investigar a divisão de responsabilidades entre software e hardware.

A contribuição técnica está na fronteira reproduzível entre os blocos: mensagens variáveis são tratadas no ARM; eventos numéricos de largura fixa são publicados em filas; o FPGA mantém estado simplificado; e a política de decisão é isolada atrás de uma interface estável. Essa organização permite verificar separadamente transporte, estado e decisão. A DE10-Nano torna possível estudar essa integração em uma placa educacional, mas as conclusões permanecem restritas ao protótipo e à configuração testada.

1.4 Estrutura do trabalho

O Capítulo 2 define os conceitos de HFT, livro de ofertas, FPGA, HLS, FAST, comunicação mapeada em memória e métricas. A partir dessa base, o Capítulo 3 compara a proposta

com estudos de microestrutura e aceleração em FPGA, enquanto o Capítulo 4 delimita a arquitetura, os contratos e a regra de decisão. O Capítulo 5 descreve materiais, procedimentos de implementação, testes e método de medição. O Capítulo 6 apresenta as evidências funcionais, os valores da campanha experimental e sua discussão. O Capítulo 7 relaciona os resultados aos objetivos e registra limitações e trabalhos futuros.

2 Referencial teórico

Este capítulo reúne apenas os conceitos usados para formular e analisar o protótipo. As decisões de implementação são apresentadas no Capítulo 4, os procedimentos no Capítulo 5 e os valores medidos no Capítulo 6.

2.1 Negociação algorítmica e HFT

Negociação algorítmica designa a produção automatizada de decisões ou ordens a partir de regras computacionais. HFT é uma classe desse campo associada a grande automação, horizontes curtos de decisão, elevado fluxo de mensagens e investimento em infraestrutura de baixa latência. Velocidade, por si só, não define um sistema completo: uma operação real também envolve dados de mercado, controle de posição e risco, roteamento de ordens e requisitos regulatórios (ALDRIDGE, 2013; HASBROUCK; SAAR, 2013).

O caminho crítico é a sequência entre a chegada de um evento relevante e a disponibilização de uma resposta. Em software de propósito geral, esse caminho pode atravessar controlador de rede, pilha de protocolos, cópias de memória, chamadas de sistema e escalonamento. Técnicas como *kernel bypass*, ou desvio da pilha de rede do núcleo do sistema operacional, e *busy polling*, ou consulta ativa de novos dados, reduzem algumas dessas etapas, mas não são usadas no protótipo deste trabalho (RIZZO, 2012; JEONG *et al.*, 2014).

Latência é o intervalo necessário para completar uma operação. Vazão é a quantidade de operações concluídas por unidade de tempo. *Jitter* é a variação da latência entre observações; uma forma simples de expressá-lo é a diferença entre o maior e o menor valor de uma amostra. Média e vazão não descrevem, sozinhas, o comportamento temporal: mediana, percentis e máximo ajudam a revelar a cauda da distribuição (JAIN, 1991). Neste trabalho, “latência interna” designa apenas o intervalo medido dentro do circuito FPGA, enquanto “latência fim a fim” designa a ida e volta entre o ARM e o FPGA.

2.2 Livro de ofertas e dados de mercado

Um livro de ofertas limitado, do inglês *limit order book*, organiza intenções de compra e venda. A melhor compra, ou *best bid*, é o maior preço comprador disponível; a melhor venda, ou *best ask*, é o menor preço vendedor. A diferença entre esses preços é o *spread*. A profundidade corresponde às ofertas disponíveis além do primeiro nível (GOULD *et al.*, 2013).

Um livro por ordem registra cada oferta e sua prioridade. Um livro por nível agrega quantidades que compartilham o mesmo preço. A segunda representação perde informações sobre ordens individuais e prioridade temporal, mas reduz o estado necessário. Em ambos os

casos, eventos de inserção, alteração, cancelamento e execução modificam o estado do livro (GOULD *et al.*, 2013; CONT; STOIKOV; TALREJA, 2010).

O desbalanceamento usado neste trabalho é a diferença entre a quantidade da melhor compra e a quantidade da melhor venda, uma simplificação de sinais baseados no estado do livro discutidos na literatura de microestrutura. Ele não representa, isoladamente, previsão de preço, intenção de negociação ou lucratividade (GOULD *et al.*, 2013; CONT; STOIKOV; TALREJA, 2010).

2.3 FPGA, SoC e HPS

Uma FPGA é um circuito integrado reconfigurável composto por blocos lógicos, registradores, memórias e interconexões programáveis. Diferentemente de um processador que executa uma sequência de instruções sobre recursos compartilhados, a FPGA implementa caminhos de dados e controle específicos. Operações independentes podem ser realizadas em paralelo quando há recursos e temporização suficientes (HARRIS; HARRIS, 2013; PATTERSON; HENNESSY, 2017).

Um sistema em chip, do inglês *System-on-Chip* (SoC), reúne diferentes subsistemas em um dispositivo. O Cyclone V SoC da DE10-Nano combina lógica FPGA e um *Hard Processor System* (HPS), isto é, um subsistema fixo com processadores ARM, controladores de memória e pontes para a lógica programável. A ponte HPS–FPGA transporta transações entre software e hardware; por isso, seu custo integra a latência do sistema, ainda que não integre a latência do núcleo lógico (Intel Corporation, 2025; Terasic Technologies, 2024; Intel Corporation, 2022).

Determinismo, neste contexto, precisa ser delimitado. Um circuito síncrono pode executar um caminho com número fixo de ciclos para as entradas aceitas, desde que não haja espera variável. Isso não implica tempo constante para uma aplicação Linux que inclui acessos a barramento, consulta de filas e escalonamento (HARRIS; HARRIS, 2013; PATTERSON; HENNESSY, 2017). O texto distingue essas duas escalas em vez de atribuir ao sistema completo uma propriedade observada apenas no circuito.

2.4 Síntese de alto nível e MATLAB HDL Coder

HLS é a transformação de uma descrição algorítmica em uma implementação no nível de transferência entre registradores, do inglês *Register Transfer Level* (RTL). O RTL explicita operações e transferências sincronizadas por relógio. Ferramentas de HLS podem gerar VHDL ou Verilog a partir de subconjuntos de C, C++ ou MATLAB, mas o projetista continua responsável por tipos, interfaces, memória, paralelismo e metas temporais (The MathWorks, Inc., 2026b).

O MATLAB HDL Coder aceita funções MATLAB e modelos Simulink compatíveis com síntese. Entradas e saídas precisam ter dimensões conhecidas; alocação dinâmica, cadeias de

caracteres e diversas bibliotecas de software não possuem correspondência direta em hardware. Aritmética inteira ou de ponto fixo torna explícita a largura dos dados e evita a inclusão não planejada de operadores de ponto flutuante (The MathWorks, Inc., 2026b; The MathWorks, Inc., 2026a).

2.5 FAST e mFAST

FAST, sigla de *FIX Adapted for STreaming*, é um padrão de codificação para fluxos de dados financeiros. Ele usa modelos de mensagem e operadores como cópia e diferença para reduzir redundância entre campos sucessivos (FIX Trading Community, 2006). mFAST é uma biblioteca C++ que codifica e decodifica mensagens segundo esses modelos (Object Computing, Inc., 2026).

O uso de FAST não torna um fluxo sintético equivalente a um fluxo de bolsa. Transporte, temporização, conjunto de eventos e distribuição estatística também compõem o comportamento dos dados de mercado (GOULD *et al.*, 2013; CONT; STOIKOV; TALREJA, 2010). O protocolo de controle de transmissão (TCP) entrega um fluxo confiável e ordenado de bytes; multicast UDP, perdas e pacotes fora de ordem pertencem a outro regime experimental e não são reproduzidos pela comunicação local adotada (EDDY, 2022).

2.6 MMIO, Avalon-MM e filas circulares

Entrada e saída mapeada em memória, do inglês *Memory-Mapped Input/Output* (MMIO), expõe registradores de um periférico no espaço de endereçamento do processador. Uma leitura ou escrita do software torna-se uma transação no barramento. Avalon-MM é a interface de memória mapeada utilizada pelo Platform Designer da Intel para conectar componentes iniciadores e periféricos (Intel Corporation, 2024a; Intel Corporation, 2024b).

Uma fila circular armazena elementos em um vetor e usa ponteiros de início e fim com retorno à primeira posição (CORMEN *et al.*, 2022). As convenções específicas de fila vazia, fila cheia, posição reservada e ordem de publicação adotadas no protótipo são decisões de projeto e, por isso, são apresentadas no Capítulo 4.

A consulta repetida de um estado é denominada *polling*. Ela simplifica o controle, mas ocupa tempo do processador. Acesso direto à memória, do inglês *Direct Memory Access* (DMA), permite transferir blocos sem uma operação da CPU para cada palavra, à custa de configuração e controle adicionais. MMIO e DMA não são alternativas universalmente superiores: a escolha depende do volume, do padrão de acesso e da latência buscada (Intel Corporation, 2022; Intel Corporation, 2019).

3 Trabalhos relacionados

Este capítulo organiza a literatura em três eixos: efeitos da baixa latência nos mercados, aceleração de funções de negociação em FPGA e representação do livro de ofertas. As decisões e os resultados do protótipo são usados apenas ao final para delimitar sua posição, sem substituir a revisão bibliográfica.

3.1 Negociação algorítmica e baixa latência

Hendershott, Jones e Menkveld (2011) analisam a relação entre negociação algorítmica e liquidez. Brogaard, Hendershott e Riordan (2014) estudam a participação de agentes de alta frequência na descoberta de preços. Hasbrouck e Saar (2013) tratam especificamente de negociação de baixa latência e mostram que o tempo de reação integra a interação estratégica entre participantes. Esses trabalhos são econômicos e empíricos; eles fundamentam a relevância do tema, mas não prescrevem uma arquitetura ARM–FPGA.

3.2 Aceleração em FPGA

Leber, Geib e Litz (2011) implementam em FPGA funções associadas ao processamento de HFT e discutem o deslocamento do caminho sensível à latência para hardware reconfigurável. Na mesma direção, Lockwood *et al.* (2012) apresentam uma biblioteca FPGA de baixa latência e destacam a integração entre rede, transporte e lógica de aplicação. Nos dois casos, a conexão da FPGA ao fluxo de rede é parte da arquitetura; isso difere de uma FPGA acessada como periférico de um processador.

Boutros *et al.* (2017) investigam HLS para construir blocos de negociação em uma FPGA Kintex UltraScale. Os autores informam 269 ns para o subsistema HFT e 869 ns no ensaio de ida e volta, valores vinculados à plataforma, à frequência e ao método descritos no artigo. Eles não constituem referência diretamente comparável ao caminho ARM–MMIO–FPGA deste trabalho.

Em conjunto, esses estudos indicam que o resultado depende não apenas do algoritmo, mas da posição do FPGA no caminho de dados e das interfaces ao redor do núcleo. A mesma literatura mostra que HLS reduz parte do trabalho de descrição RTL sem eliminar decisões sobre largura, paralelismo e protocolo (LEBER; GEIB; LITZ, 2011; LOCKWOOD *et al.*, 2012; BOUTROS *et al.*, 2017).

3.3 Livro de ofertas

Gould *et al.* (2013) revisam a estrutura e a dinâmica de livros de ofertas, incluindo preço, prioridade, profundidade e tipos de evento. Cont, Stoikov e Talreja (2010) modelam chegadas,

cancelamentos e execuções como processos que atualizam um estado discreto. Essas representações abrangem fenômenos que o protótipo não reproduz, como ordens individuais e dinâmica estatística de um mercado.

Na implementação em hardware, a representação por nível e com profundidade fixa reduz estruturas de busca e torna conhecidos os limites dos laços. Essa simplificação é compatível com validação de integração, mas não autoriza tratar o estado resultante como reconstrução completa de um livro real (GOULD *et al.*, 2013; BOUTROS *et al.*, 2017).

3.4 Posicionamento do protótipo

O Quadro 1 explicita a diferença de escopo entre a literatura e o protótipo. O quadro não estabelece um ranking de desempenho; ele identifica quais aspectos foram preservados e quais foram excluídos.

Quadro 1 – Relação entre a literatura e o escopo do protótipo

Eixo	Literatura revisada	Escopo deste trabalho
Mercado	Liquidez, descoberta de preços e interação entre participantes.	Contexto de aplicação; não há análise econômica ou de lucratividade.
Fluxo de dados	Redes e protocolos integrados ao caminho de baixa latência.	Fluxo sintético por TCP, decodificado em software.
FPGA	Núcleos e bibliotecas conectados a redes especializadas.	FPGA acessada pelo ARM por MMIO em placa educacional.
Livro de ofertas	Ordens ou níveis, múltiplos eventos e modelos estatísticos.	Cinco símbolos do gerador, níveis agregados e profundidade de oito posições por lado.
HLS	Geração de hardware condicionada pela arquitetura e pelos tipos.	Geração apenas da regra de decisão; ponte e livro permanecem em VHDL manual.
Validação	Latência, vazão, área e comparação conforme cada plataforma.	Testes funcionais e uma campanha inicial de latência e vazão na DE10-Nano.

Fonte: Elaborado pelos autores com base em Leber, Geib e Litz (2011), Lockwood *et al.* (2012), Boutros *et al.* (2017), Gould *et al.* (2013) e Cont, Stoikov e Talreja (2010).

A contribuição do protótipo é de integração e documentação: ele reúne fluxo sintético, receptor C++, filas MMIO, livro por nível e estratégia gerada a partir de MATLAB. Não se reivindica desempenho equivalente ao dos sistemas relacionados nem substituição de infraestrutura de negociação profissional.

4 Proposta

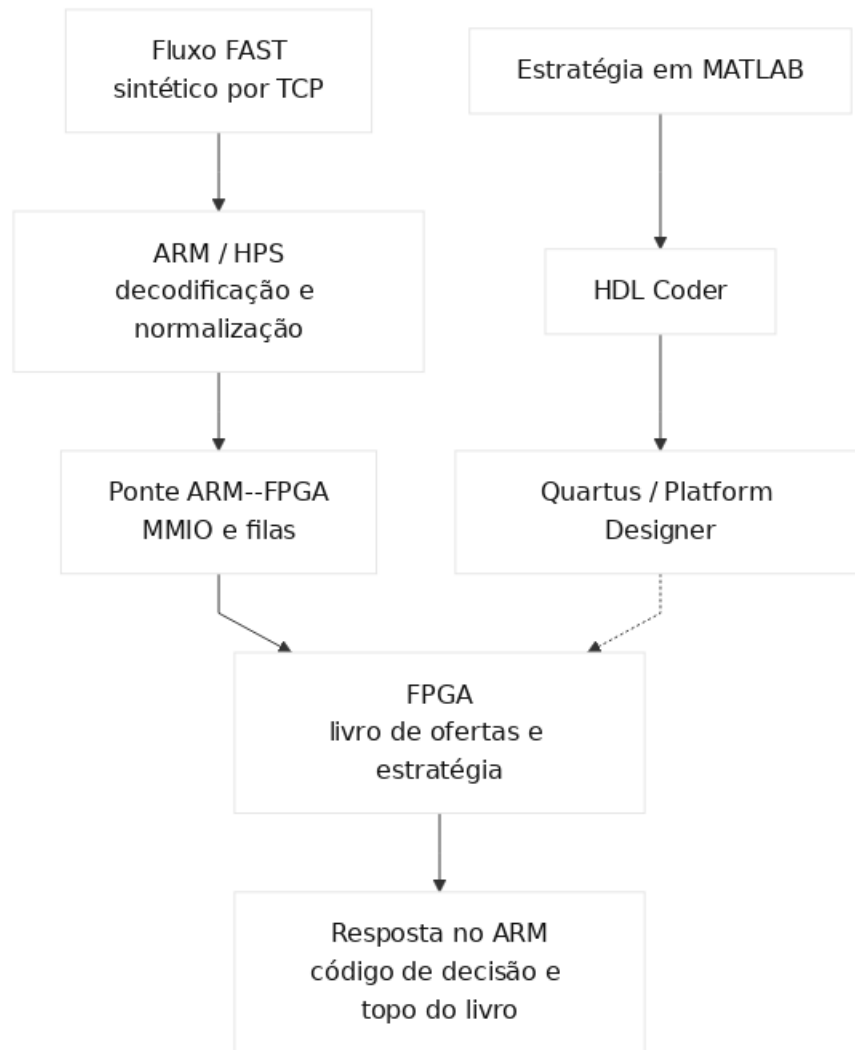
Este capítulo define o que foi proposto antes de apresentar como a implementação foi conduzida ou quais resultados foram obtidos. Trata-se de um protótipo acadêmico para exercitar o caminho entre dados sintéticos, software ARM e lógica FPGA. A proposta não inclui conexão a bolsa nem emissão de ordens.

4.1 Arquitetura e fronteiras

A arquitetura separa o sistema em três domínios complementares. No software, um gerador publica mensagens FAST sintéticas por TCP e um receptor C++ as decodifica; quando o acesso ao FPGA está habilitado, esse receptor converte símbolo, lado, preço e quantidade para palavras inteiras. Na lógica programável, duas filas transportam eventos e respostas, um livro de ofertas mantém níveis por símbolo e uma estratégia produz um código de decisão. No fluxo de desenvolvimento, o HDL Coder gera o VHDL da estratégia a partir de uma função MATLAB.

A Figura 2 apresenta o percurso de execução e, pela ligação tracejada, o percurso de geração. Durante a execução, os eventos seguem do fluxo sintético para o ARM, atravessam a ponte MMIO e chegam ao FPGA; a resposta retorna ao receptor. MATLAB e HDL Coder não participam desse percurso em tempo de execução: eles produzem um arquivo VHDL que é incorporado ao projeto antes da síntese.

Esse percurso define a responsabilidade de cada bloco: o ARM trata protocolo e conversão de representação; a ponte e as filas implementam o contrato binário; o livro mantém o estado simplificado; e a estratégia devolve NOOP, BUY ou SELL. Esses códigos são saídas internas de teste, não ordens transmitidas a um mercado. A arquitetura cria três fronteiras verificáveis: bytes FAST tornam-se campos C++ tipados; esses campos tornam-se um quadro de oito palavras de 32 bits; e a atualização do livro produz os sinais de topo consumidos pela estratégia.

Figura 2 – Arquitetura geral do protótipo

Fonte: Elaborado pelos autores.

4.2 Decisões arquiteturais

A decodificação FAST permanece no ARM porque envolve biblioteca, modelo de mensagem, campos textuais e tratamento de conexão. Com isso, o FPGA recebe uma representação já normalizada. A escolha reduz o escopo do hardware, mas mantém o sistema operacional no caminho de execução e afasta o protótipo de arquiteturas em que a FPGA recebe quadros diretamente da rede.

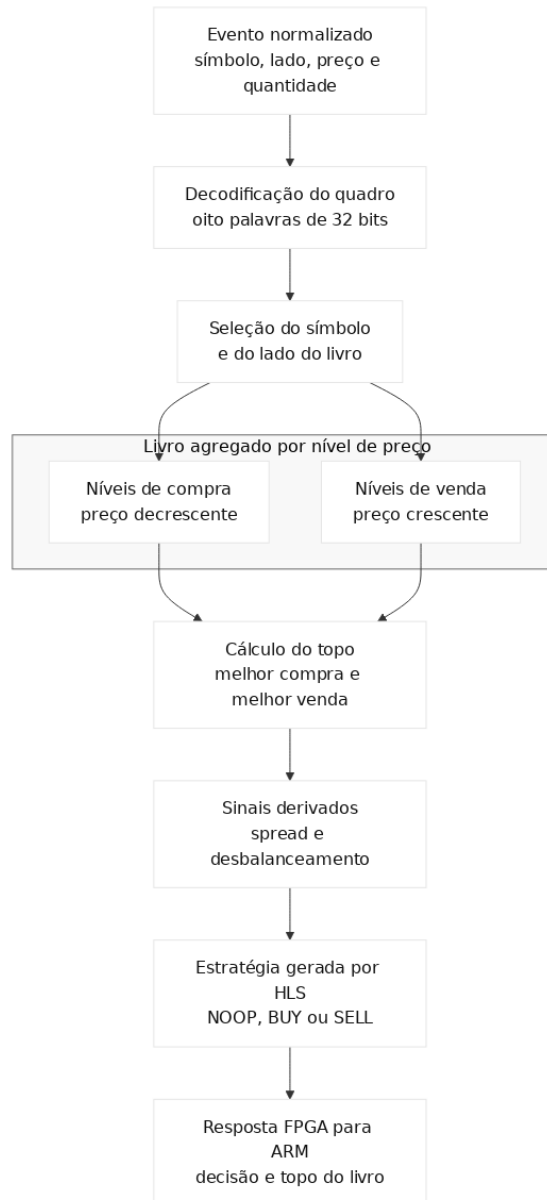
A comunicação usa duas filas circulares em MMIO: TX, do ARM para o FPGA, e RX, no sentido inverso. As filas desacoplam produtor e consumidor enquanto houver posições livres. A capacidade útil é uma posição menor que a profundidade física, e a publicação dos ponteiros só ocorre depois da escrita ou leitura completa do quadro.

Livro e estratégia são tratados como blocos distintos. O livro mantém estado e ordena até oito níveis de compra e oito de venda por símbolo; a estratégia, por sua vez, é combinacional

e recebe melhor compra, melhor venda, respectivas quantidades, *spread* e desbalanceamento. Uma alteração da regra exige nova geração, novos testes e nova síntese do projeto, mas não exige mudar o formato do evento enquanto a interface do bloco for preservada.

A Figura 3 mostra essa separação. O quadro é decodificado e direcionado ao símbolo e ao lado correspondentes; depois da atualização, os sinais do topo alimentam a estratégia e integram a resposta. A figura representa o fluxo lógico, não a temporização em ciclos.

Figura 3 – Fluxo lógico do livro de ofertas simplificado



Fonte: Elaborado pelos autores.

O ponto substituível da proposta é a função `strategy.m`, com seis entradas inteiras e uma saída de decisão. A substituição preserva a interface com o livro, mas exige gerar novamente o VHDL, executar os testes e recompilar o projeto FPGA. O procedimento correspondente é descrito na seção 5.2.

4.3 Contrato ARM-FPGA

Cada posição das filas contém oito palavras de 32 bits. A Tabela 1 apresenta o quadro de entrada. Os campos 6 e 7 são reservados e valem zero na implementação atual. O evento de reinicialização limpa o livro do símbolo indicado; o receptor do fluxo sintético publica eventos do tipo inserção ou atualização.

Tabela 1 – Quadro enviado do ARM para o FPGA

Palavra	Campo	Representação	Semântica
0	seq_no	sem sinal, 32 bits	Número de sequência sintético.
1	symbol_id	sem sinal, 32 bits	Identificador numérico do símbolo.
2	price_1e4	sem sinal, 32 bits	Preço multiplicado por 10^4 .
3	qty	sem sinal, 32 bits	Quantidade agregada no nível.
4	event_type	sem sinal, 32 bits	1: inserir/atualizar; 2: remover; 3: reinicializar.
5	side	sem sinal, 32 bits	1: compra; 2: venda.
6	reservado	32 bits	Reservado para extensão.
7	reservado	32 bits	Reservado para extensão.

Fonte: Elaborado pelos autores conforme o contrato implementado em C++ e VHDL.

A Tabela 2 descreve a resposta. O campo `action` é um código interno de decisão. BUY e SELL indicam qual ramo da regra foi ativado; nenhum dos dois provoca transmissão de ordem.

Tabela 2 – Quadro devolvido do FPGA para o ARM

Palavra	Campo	Representação	Semântica
0	seq_no	sem sinal, 32 bits	Sequência ecoada para correlação.
1	action	sem sinal, 32 bits	0: NOOP; 1: BUY; 2: SELL.
2	best_bid_px	sem sinal, 32 bits	Melhor preço de compra, em escala 10^4 .
3	best_bid_qty	sem sinal, 32 bits	Quantidade da melhor compra.
4	best_ask_px	sem sinal, 32 bits	Melhor preço de venda, em escala 10^4 .
5	best_ask_qty	sem sinal, 32 bits	Quantidade da melhor venda.
6	spread_1e4	sem sinal, 32 bits	Melhor venda menos melhor compra.
7	imbalance	com sinal, 32 bits	Quantidade compradora menos vendedora.

Fonte: Elaborado pelos autores conforme o contrato implementado em C++ e VHDL.

O FPGA informa assinatura, versão, profundidades e número de palavras por posição, e o software usa essa geometria para calcular a base da fila RX por

$$RX_BASE = TX_BASE + TX_DEPTH \times SLOT_WORDS \times 4. \quad (1)$$

Com profundidade 64, oito palavras e base TX 0×100 , a base RX é 0×900 . O cálculo evita vincular o software a uma única geometria.

4.4 Símbolos, regra de decisão e delimitação

O receptor converte os cinco símbolos do gerador conforme a Tabela 3. Símbolos não mapeados são descartados apenas no caminho para o FPGA; a tabela é uma convenção do protótipo, não uma codificação de mercado.

Tabela 3 – Mapeamento de símbolos do fluxo sintético

Símbolo	Identificador	Símbolo	Identificador
AAPL	0	MSFT	1
NVDA	2	GOOGL	3
TSLA	4		

Fonte: Elaborado pelos autores conforme `fast_receiver.cpp`.

A regra retorna NOOP se um lado estiver vazio, se a melhor venda não superar a melhor compra ou se o *spread* exceder 2,5000 unidades monetárias. Nos demais casos, retorna BUY para desbalanceamento maior ou igual a 500, SELL para valor menor ou igual a -500 e NOOP no intervalo restante. O Listagem 1 explicita a ordem dessas condições e os valores de retorno implementados em `strategy.m`.

Listagem 1 – Pseudocódigo da regra de decisão

```

1 FUNCAO decidir(melhor_compra, qtd_compra,
2               melhor_venda, qtd_venda,
3               spread_1e4, desbalanceamento)
4   acao <- NOOP
5
6   SE qtd_compra = 0 OU qtd_venda = 0 ENTAO
7     RETORNE acao
8   FIM SE
9
10  SE melhor_venda <= melhor_compra ENTAO
11    RETORNE acao
12  FIM SE
13
14  SE spread_1e4 > 25000 ENTAO
15    RETORNE acao
16  FIM SE
17
18  SE desbalanceamento >= 500 ENTAO
19    acao <- BUY
20  SENAO SE desbalanceamento <= -500 ENTAO
21    acao <- SELL
22  FIM SE
23
24  RETORNE acao
25 FIM FUNCAO

```

Fonte: Elaborado pelos autores conforme `matlab/strategy.m`.

Os limiares foram escolhidos pelos autores para produzir ramos verificáveis com o fluxo sintético; não foram calibrados com dados históricos e não devem ser interpretados como estratégia financeira.

O receptor possui dois modos de execução. Sem a variável `HFT_FPGA_MMIO_BASE`, ele opera somente em software: conecta-se ao gerador, decodifica e imprime as mensagens. Com a variável definida, também abre a região MMIO, confere assinatura e geometria e publica eventos na fila TX. A separação entre os modos ajuda a distinguir problemas de protocolo de problemas da integração física.

O escopo termina na impressão da resposta. Não há canal de envio de ordens, conexão com bolsa, dados históricos, controle de posição, gestão de risco, multicast, carimbo de tempo de rede ou avaliação de resultado financeiro. Desse modo, a execução funcional valida a cadeia implementada, não uma operação de HFT completa.

5 Materiais e métodos

Este capítulo descreve os recursos empregados, o procedimento de implementação, os testes funcionais e o método da campanha quantitativa. Valores obtidos são reservados ao Capítulo 6.

5.1 Materiais

O alvo físico foi a DE10-Nano, que contém um Cyclone V SoC com processadores ARM Cortex-A9 e lógica FPGA (Terasic Technologies, 2024; Intel Corporation, 2025). O Quadro 2 relaciona as ferramentas ao papel exercido no trabalho.

Quadro 2 – Materiais empregados no desenvolvimento

Material	Uso no trabalho
DE10-Nano	Execução do software ARM e da lógica sintetizada no Cyclone V.
C++ e mFAST	Gerador, receptor, normalização, acesso MMIO e programa de medição.
VHDL e GHDL	Ponte, filas, livro, invólucros, motor integrado e simulações.
MATLAB HDL Coder	Descrição, teste e geração VHDL da regra de decisão.
Quartus Prime Lite e Platform Designer	Síntese e integração do periférico ao HPS.
Make, CMake e Docker	Automação das compilações e dos testes que não dependem da placa ou da licença MATLAB.

Fonte: Elaborado pelos autores.

Os arquivos-fonte, as rotinas de construção, os bancos de teste e os arquivos LaTeX deste documento estão disponíveis no repositório público <https://github.com/forestileao/hft-matlab-fpga>. A identificação dos comandos e arquivos ao longo do texto corresponde à organização versionada nesse endereço; quando citados como caminhos ou alvos de construção, eles são referenciados por links para o próprio repositório. As indicações entre parênteses registram a função do artefato para preservar a leitura também em cópia impressa.

5.2 Procedimento de implementação

O desenvolvimento foi dividido em interfaces verificáveis. A sequência partiu da configuração do gerador e do receptor com o modelo `FAST cpp/src/templates/SimpleMD.xml` (modelo de mensagem FAST); a partir desse fluxo, definiu-se o quadro de oito palavras e implementaram-se as filas circulares. O livro por nível passou a consumir esse quadro e a produzir sinais de topo, que alimentaram a função MATLAB gerada para VHDL e ligada ao motor por um invólucro com controle de validade e prontidão.

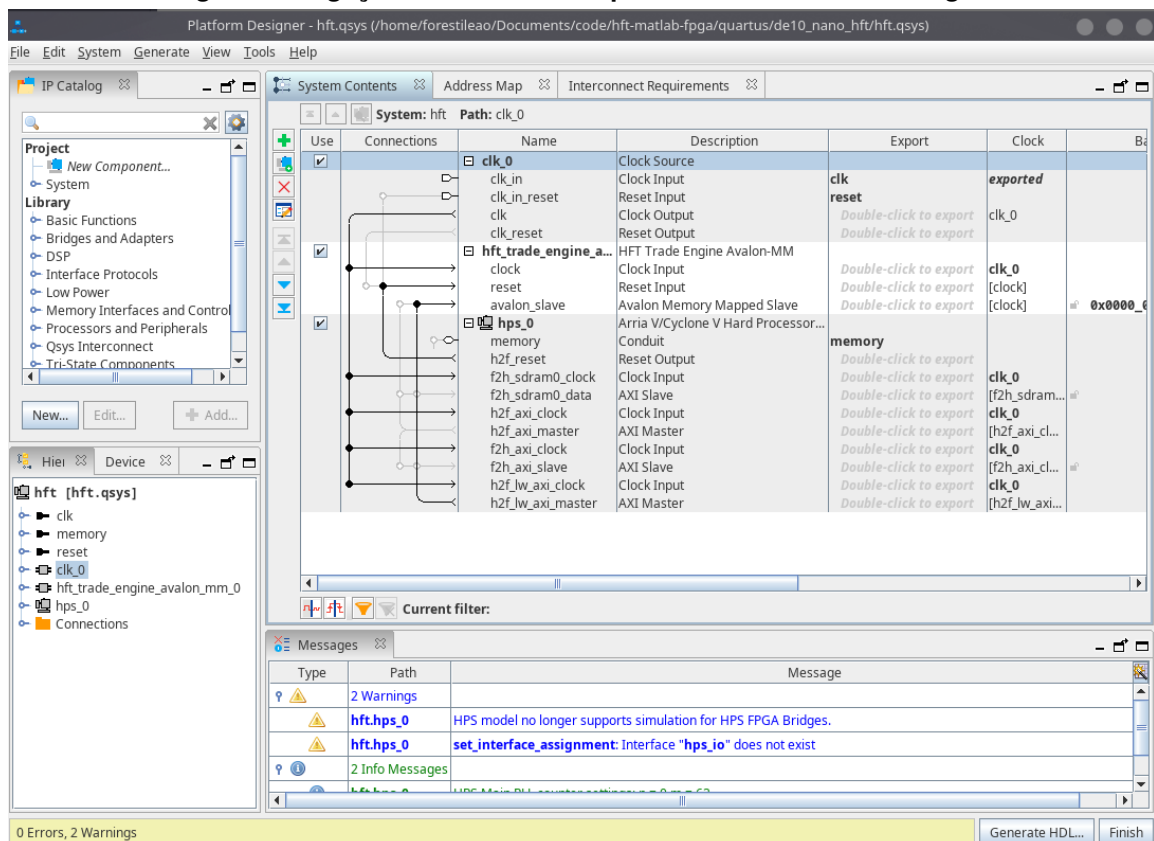
O projeto usa inteiros sem sinal de 32 bits para preços e quantidades. O preço é multiplicado por 10^4 antes do envio, e o desbalanceamento é tratado como inteiro com sinal de 32 bits. Foram configurados oito símbolos possíveis no núcleo e oito níveis por lado; o gerador utiliza cinco desses identificadores.

5.2.1 Integração no Platform Designer

O periférico `hft_trade_engine_avalon_mm` (interface Avalon-MM do motor) adapta Avalon-MM à interface interna do motor. O HPS inicia leituras e escritas pela ponte leve HPS–FPGA, e o periférico expõe cabeçalho, registradores, contadores e regiões das filas (Intel Corporation, 2024a; Intel Corporation, 2024b; Intel Corporation, 2022).

A ligação montada no Platform Designer é mostrada na Figura 4. O aspecto relevante é a conexão do HPS ao periférico customizado; a captura não demonstra, isoladamente, que o caminho funcional ou temporal esteja correto. Essas propriedades foram avaliadas pelos testes e pela execução descritos adiante.

Figura 4 – Ligação entre o HPS e o periférico no Platform Designer



Fonte: Captura de tela produzida pelos autores no Quartus Prime Lite.

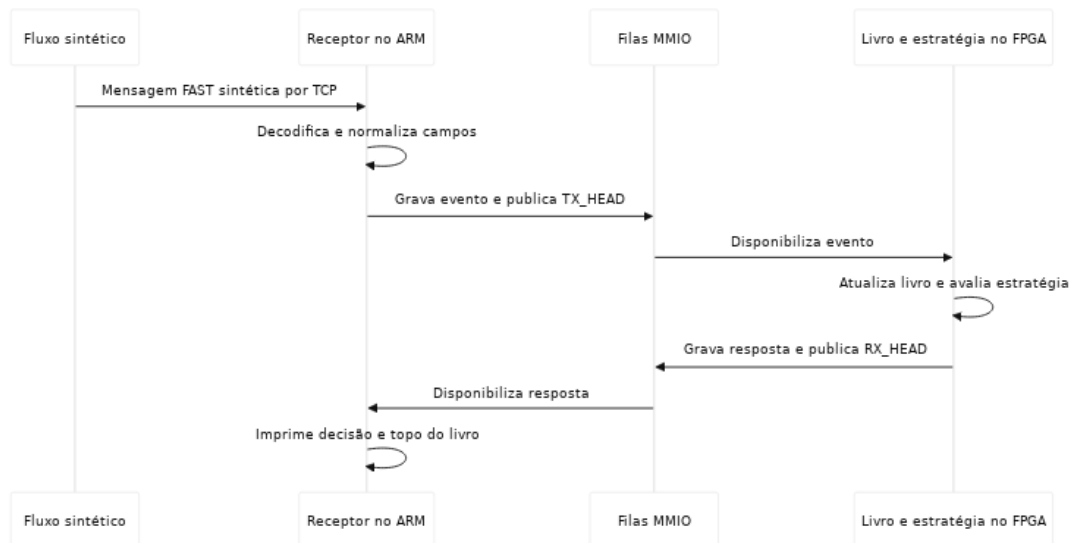
5.2.2 Fluxo sintético de dados

O programa `fast_data_feed` (gerador sintético) escolhe pseudoaleatoriamente um dos cinco símbolos, um lado, uma quantidade entre 100 e 5.000 e um deslocamento de preço. Uma mensagem contém um único nível e é enviada a cada 200 ms por TCP na porta 9001. O programa `fast_receiver` (receptor e normalizador) decodifica essa mensagem com mFAST. No modo com FPGA, ele mapeia o símbolo, converte o preço e publica um evento do tipo inserção ou atualização.

Esse procedimento permitiu verificar codificação, decodificação e integração. A taxa fixa, a distribuição simples e a ausência de cancelamentos e execuções impedem interpretá-lo como reprodução estatística de um mercado.

A sequência do ensaio funcional está organizada na Figura 5. O gerador envia a mensagem ao receptor; o ARM decodifica e publica o evento; o FPGA atualiza o livro e avalia a regra; e a resposta retorna pela fila RX. Essa figura descreve o procedimento, não a região temporizada pelo programa de medição.

Figura 5 – Fluxo de um evento no ensaio funcional



Fonte: Elaborado pelos autores.

5.2.3 Geração e integração da estratégia

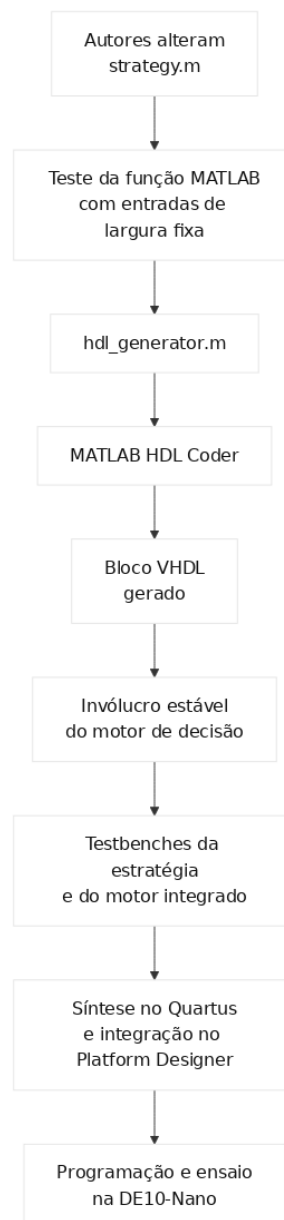
A integração da função `matlab/strategy.m` (regra de decisão em MATLAB) seguiu as restrições de geração descritas na documentação do HDL Coder (The MathWorks, Inc., 2026b) e foi conduzida pelo seguinte procedimento:

1. definição da função MATLAB com seis entradas inteiras e uma saída de decisão;
2. execução dos testes MATLAB da regra;

3. execução de `matlab/hdl_generator.m` (roteiro de geração HLS) para gerar o VHDL pelo HDL Coder;
4. conexão do arquivo gerado ao invólucro VHDL do motor;
5. execução dos bancos de teste da estratégia e do motor integrado;
6. recompilação do projeto no Quartus e programação da DE10-Nano.

A Figura 6 representa esse procedimento. A geração do arquivo VHDL é uma etapa intermediária, de modo que os testes e a análise de temporização continuam necessários depois de cada alteração.

Figura 6 – Procedimento de alteração e geração da estratégia



Fonte: Elaborado pelos autores.

5.3 Validação funcional

A validação foi organizada por camada para separar responsabilidades e falhas possíveis. O Quadro 3 apresenta o artefato, o comportamento observado e o comando reproduzível. “Validado”, neste contexto, significa que os casos codificados nos testes foram executados sem asserções de falha; não significa cobertura exaustiva nem certificação operacional.

Quadro 3 – Procedimentos de validação funcional

Camada	Comportamentos exercitados	Procedimento
Invólucro C++	Geometria, base RX, fila cheia/vazia, envio e recepção.	<code>make cpp-test.</code>
Ponte VHDL	Registradores, ordem das palavras, retorno de índice, rajadas e contrapressão.	<code>make vhdl-test</code> e <code>make vhdl-test-fast.</code>
Livro	Inserção, atualização, remoção, ordenação e rejeição de cruzamento.	<code>make vhdl-test-order-book.</code>
Estratégia gerada	Ramos NOOP, BUY e SELL e empacotamento da resposta.	<code>make vhdl-test-strategy.</code>
Motor integrado	Caminho entre fila, livro, estratégia e resposta.	<code>make vhdl-test-engine.</code>
Avalon-MM	Leituras e escritas do periférico e conclusão das transações.	<code>make vhdl-test-avalon.</code>
Fluxo e receptor	Codificação, transmissão TCP e decodificação sem FPGA.	<code>make cpp-smoke.</code>

Fonte: Elaborado pelos autores a partir dos alvos do `Makefile` (automação de construção e testes) e dos casos de teste do repositório.

Os testes automatizados foram complementados por um ensaio funcional na placa. O critério foi reconhecer assinatura e geometria da ponte, publicar eventos do fluxo sintético e observar respostas com a mesma sequência e campos coerentes com o estado do livro. Esse ensaio verifica integração no cenário executado, mas não substitui a campanha de desempenho nem testes com dados reais.

5.4 Método da avaliação quantitativa

O programa `fpga_benchmark` (ensaio quantitativo) implementa o ensaio de desempenho e foi usado separadamente do fluxo FAST. Ele pré-constrói eventos determinísticos e, durante a região medida, não executa TCP, decodificação FAST, esperas de 200 ms, geração aleatória ou impressão por mensagem. Essa separação permite medir o núcleo e a interface MMIO, mas impede atribuir os valores ao fluxo completo desde a rede.

A campanha registrada utilizou a lógica a 50 MHz, 10.000 eventos de aquecimento e 1.000.000 de eventos medidos. O programa pré-constrói 1.010.000 eventos: os índices 0 a

9.999 aquecem o estado, e os índices 10.000 a 1.009.999 integram a região medida. Para um índice inteiro i , símbolo, lado, deslocamento de preço e quantidade são definidos por

$$s_i = i \bmod 5, \quad (2)$$

$$c_i = \begin{cases} \text{compra}, & i \bmod 2 = 0, \\ \text{venda}, & i \bmod 2 = 1, \end{cases} \quad (3)$$

$$d_i = 100 [((17i) \bmod 80) + 1], \quad (4)$$

$$q_i = 100 + ((37i) \bmod 4901), \quad (5)$$

$$p_i = \begin{cases} b_{s_i} - d_i, & c_i = \text{compra}, \\ b_{s_i} + d_i, & c_i = \text{venda}, \end{cases} \quad (6)$$

em que b_{s_i} é o preço-base do símbolo, com vetor (1 850 000, 4 150 000, 8 750 000, 1 700 000, 1 750 000) para AAPL, MSFT, NVDA, GOOGL e TSLA, respectivamente, e os preços são representados na escala 10^4 . A sequência é $i + 1$. Como 17 e 80 são coprimos, os 80 deslocamentos aparecem com a mesma frequência em cada período; como 37 e 4901 também são coprimos, as 4901 quantidades percorrem todo o conjunto antes de se repetir. Nos 1.000.000 de eventos medidos, cada símbolo aparece 200.000 vezes, cada lado 500.000 vezes, cada deslocamento 12.500 vezes e cada quantidade aparece 204 ou 205 vezes.

Do ponto de vista estocástico, a carga pode ser descrita por sua distribuição empírica: cada símbolo recebe frequência relativa de 20%, cada lado recebe 50% e cada deslocamento de preço recebe 1,25%. As 4901 quantidades aparecem com frequência relativa de 0,0204% ou 0,0205%, o que torna as marginais uniformes ou aproximadamente uniformes. Essa caracterização, porém, não transforma os campos em amostras aleatórias independentes, pois todos derivam do mesmo índice e a distribuição conjunta não corresponde ao produto das marginais. Também não há processo aleatório para intervalos de chegada, cancelamentos ou execuções. A carga é adequada para repetir o mesmo ensaio computacional, mas não constitui um modelo estocástico de mercado (CONT; STOIKOV; TALREJA, 2010; GOULD *et al.*, 2013).

Para reproduzir o conjunto e a ordem dos eventos, utiliza-se a mesma revisão de `cpp/src/fpga_benchmark.cpp` (geração determinística dos eventos) e executa-se `fpga_benchmark -mode full -messages 1000000 -warmup 10000`. A função de geração depende apenas do índice inteiro; não recebe relógio, estado externo nem semente pseudoaleatória. Com isso, os 10.000 eventos de aquecimento e os 1.000.000 de eventos medidos são recriados na mesma ordem. Essa reprodutibilidade caracteriza a carga

de entrada e não implica que os tempos medidos serão idênticos entre execuções. O programa executa três modos:

- `sw-core`: executa no ARM uma implementação C++ do livro e da regra, sem entrada e saída por mensagem;
- `fpga-mmio`: mantém múltiplos eventos em trânsito nas filas e mede a vazão do caminho ARM-FPGA-ARM;
- `fpga-sync`: envia um evento, aguarda a respectiva resposta e registra o tempo de ida e volta antes de prosseguir.

No núcleo FPGA, um contador registra os ciclos entre a aceitação do comando e a produção da resposta. O período nominal é 20 ns a 50 MHz. São coletados mínimo, máximo, soma e contagem; a média é calculada pela razão entre soma e contagem, e a variação interna pela diferença entre máximo e mínimo.

No ARM, o relógio monotônico bruto do sistema, quando disponível, mede o tempo. No modo síncrono, uma amostra envolve a chamada de envio, as consultas até haver resposta e a leitura da resposta; depois, o programa ordena as amostras e calcula média, mediana, percentil 99, mínimo e máximo. No modo em fluxo, a vazão é a contagem de respostas dividida pelo intervalo total. A referência C++ é a duração do laço dividida por um milhão de eventos. O fator de aceleração do núcleo é a média C++ dividida pela média interna do FPGA; ele não inclui MMIO.

Os números preservados no texto correspondem a uma campanha. Não há, no repositório, arquivo bruto com a saída JSON dessa execução nem séries de campanhas independentes. Os resultados caracterizam a execução registrada e não fornecem intervalo de confiança ou análise de repetibilidade entre inicializações. A energia elétrica não foi instrumentada; por essa razão, não integra as métricas medidas e é tratada separadamente como experimento de cálculo.

O Quadro 4 resume o significado operacional de cada métrica e evita comparar grandezas de escopos diferentes.

Quadro 4 – Métricas e respectivos escopos de medição

Métrica	Procedimento	Inclui
Latência interna FPGA	Contador de ciclos entre comando aceito e resposta produzida.	Livro, estratégia e controle interno; exclui MMIO e ARM.
Tempo do núcleo C++	Tempo total do laço <code>sw-core</code> dividido pelos eventos.	Algoritmo equivalente no ARM; exclui TCP e MMIO.
Latência fim a fim	Amostras do modo <code>fpga-sync</code> .	Chamadas de fila, MMIO, espera ativa e núcleo FPGA.
Vazão	Respostas por segundo no modo <code>fpga-mmio</code> .	Transferência pelas duas filas e processamento FPGA.
Variação	Máximo menos mínimo no escopo correspondente.	Resolução e fontes de variação próprias de cada medição.

Fonte: Elaborado pelos autores conforme `cpp/src/fpga_benchmark.cpp` (programa de medição).

5.5 Método do experimento de cálculo energético

A energia por decisão foi tratada como experimento de cálculo porque não houve instrumentação elétrica na placa. Para cada substrato, aplicou-se

$$E = P \times t, \quad (7)$$

em que E é a energia estimada por decisão, P é uma potência de referência e t é o tempo atribuído ao núcleo lógico ou computacional. Foram adotadas as hipóteses de 50 mW para a parcela dinâmica da lógica Cyclone V, 1,5 W para um núcleo ARM Cortex-A9 e 15 W para um núcleo x86. Para os tempos, usaram-se os valores registrados de 60 ns no FPGA e 315 ns no ARM, além de 30 ns como hipótese ilustrativa para o núcleo x86.

Essas potências não foram medidas no mesmo limite físico: uma representa a lógica FPGA, outra o núcleo ARM e outra um núcleo de servidor. O tempo x86 também não foi obtido no experimento. O cálculo serve para comparar ordens de grandeza sob as hipóteses declaradas; ele não mede o consumo da DE10-Nano, não inclui comunicação e não permite concluir eficiência energética fim a fim.

6 Resultados e discussão

Este capítulo apresenta três conjuntos de evidências: execução funcional na placa, testes automatizados e uma campanha quantitativa. Cada conjunto responde a uma pergunta diferente. A execução em placa mostra que os componentes foram integrados; os testes exercitam casos especificados; e a campanha caracteriza tempo e vazão na configuração registrada. Nenhum deles avalia lucratividade, envio de ordens ou operação em bolsa.

6.1 Execução funcional na DE10-Nano

No ensaio preservado pelos autores, o receptor foi iniciado com base MMIO `0xff200000` e reconheceu assinatura `0x48465431`, versão 1, profundidade 64, oito palavras por posição e base RX `0x900`. Após a conexão ao gerador local, os eventos produziram respostas com a mesma sequência.

O Listagem 2 contém três observações desse ensaio. Nos dois primeiros casos, a diferença entre as quantidades do topo excede 500 e a regra do Listagem 1 retorna BUY. No terceiro, a diferença é 121 e a saída é NOOP. O trecho evidencia correlação e coerência com a regra; por ser uma amostra curta, não mede taxa de erro nem cobre todos os eventos possíveis.

Listagem 2 – Trecho da saída do receptor com respostas FPGA–ARM

```

1 FPGA MMIO bridge enabled: base=0xff200000 span=0x2000
2 magic=0x48465431 version=1 tx_depth=64 rx_depth=64
3 slot_words=8 rx_base=0x900 mode=new
4 Connected to 127.0.0.1:9001
5
6 seq=26 sym=TSLA side=sell price=175.17 qty=1547
7 [FPGA->ARM] seq=26 action=BUY
8   best_bid_px_1e4=1749900 best_bid_qty=2836
9   best_ask_px_1e4=1750100 best_ask_qty=681
10  spread_1e4=200 imbalance=2155
11
12 seq=27 sym=NVDA side=sell price=875.13 qty=3861
13 [FPGA->ARM] seq=27 action=BUY
14   best_bid_px_1e4=8749800 best_bid_qty=3282
15   best_ask_px_1e4=8750100 best_ask_qty=2259
16   spread_1e4=300 imbalance=1023
17
18 seq=36 sym=AAPL side=sell price=185.75 qty=1795
19 [FPGA->ARM] seq=36 action=NOOP
20   best_bid_px_1e4=1849900 best_bid_qty=939
21   best_ask_px_1e4=1850300 best_ask_qty=818
22   spread_1e4=400 imbalance=121

```

Fonte: Registro de execução dos autores na DE10-Nano.

A Tabela 4 relaciona cada conclusão funcional à evidência disponível e explicita o limite do ensaio, que não inclui ordem de saída, bolsa externa ou instrumento financeiro real.

Tabela 4 – Evidências do ensaio funcional na DE10-Nano

Item	Evidência observada
Identificação	Assinatura, versão e geometria lidas antes dos eventos.
Comunicação	Conexão TCP local e sequências correlacionadas nas respostas.
Estado	Melhores preços, quantidades, <i>spread</i> e desbalanceamento devolvidos.
Decisão	BUY e NOOP coerentes com os limiares nos exemplos registrados.
Limite	Não foram exercitados envio de ordens, risco ou conexão externa.

Fonte: Elaborado pelos autores a partir do registro do Listagem 2.

6.2 Resultados dos testes automatizados

Os procedimentos do Quadro 3 foram executados na versão revisada. Os dois testes C++ concluíram sem falha: o teste do invólucro e o teste rápido do núcleo de referência. Também concluíram os seis alvos VHDL: ponte básica, ponte com rajadas e retorno de índices, livro, estratégia gerada, motor e periférico Avalon-MM.

Os bancos de teste que instanciam a estratégia gerada emitiram avisos de `NUMERIC_STD` no instante inicial, antes de os sinais assumirem valores conhecidos, mas alcançaram suas asserções finais de sucesso. Os avisos indicam uma oportunidade de inicialização mais limpa nos testes; não houve asserção funcional de falha. A evidência permanece limitada aos casos neles codificados.

O Quadro 5 resume a quantidade de testes concluídos em cada grupo e registra execução sem falha, não porcentagem de cobertura do código.

Quadro 5 – Resultado da execução dos testes automatizados

Grupo	Escopo	Resultado
C++	Invólucro MMIO e núcleo C++ de referência.	2 de 2 testes concluídos.
Ponte VHDL	Registradores, filas, rajadas e contrapressão.	2 bancos de teste concluídos.
Processamento VHDL	Livro, estratégia gerada e motor integrado.	3 bancos de teste concluídos.
Avalon-MM	Acesso ao periférico pela interface de barramento.	1 banco de teste concluído.

Fonte: Execução dos alvos `make cpp-test` e `make vhd1-test-all`.

6.3 Campanha quantitativa

Os valores desta seção foram transcritos de uma campanha com 1.000.000 de eventos, precedidos de 10.000 eventos de aquecimento, na DE10-Nano a 50 MHz. A saída JSON bruta

não está versionada; por isso, a análise não pode recalcular estatísticas nem avaliar variação entre execuções independentes. Os números são reportados com esse limite.

A Tabela 5 reúne as métricas registradas. A latência interna é medida pelo contador no FPGA. O tempo C++ mede somente o algoritmo de referência. A latência fim a fim mede o modo síncrono entre chamadas de envio e recepção no ARM. A vazão usa o modo em fluxo, com mais de um evento em trânsito.

Tabela 5 – Valores registrados na campanha quantitativa

Métrica	Escopo	Valor
Latência interna FPGA	Comando aceito até resposta produzida.	3 ciclos (60 ns)
Variação interna	Máximo menos mínimo do contador de ciclos.	Não resolvida; desprezível na resolução de 1 ciclo (20 ns)
Tempo médio C++	Livro e regra no ARM, sem comunicação.	315 ns/evento
Razão C++/FPGA	Razão reportada entre as médias não arredondadas.	5,253 vezes
RTT médio	ARM-MMIO–FPGA-MMIO–ARM, modo síncrono.	10.228 ns
RTT p99	Percentil 99 do mesmo modo.	10.310 ns
RTT máximo	Maior amostra registrada.	1.146.360 ns
Amplitude do RTT	Máximo menos mínimo registrado.	1.136.220 ns
Vazão	Modo em fluxo pelas filas MMIO.	102.381 mensagens/s

Fonte: Campanha dos autores com `fpga_benchmark -mode full -messages 1000000 -warmup 10000`.

O programa calcula também mediana, mas esse valor não foi preservado no texto nem em arquivo bruto. Percentis 90 e 95 tampouco foram registrados. Acrescentá-los agora exigiria repetir a campanha na placa; não é possível derivá-los dos valores disponíveis.

6.3.1 Variação interna e resolução do contador

O contador atribuiu três ciclos tanto ao mínimo quanto ao máximo para os eventos da campanha. Como sua resolução é de um ciclo, ou 20 ns, o experimento não distingue variações menores que esse intervalo. A variação interna é tratada como desprezível na resolução adotada. Essa classificação vale apenas para o núcleo e para a configuração testados; ela não determina a variação de outra implementação, frequência, condição de espera ou estratégia regenerada.

O núcleo possui caminhos sincronizados e tamanhos fixos, o que explica a contagem constante nos casos medidos. Essa observação não elimina outras fontes de variação: o atendimento da frequência depende do relatório de temporização da síntese, e o sistema completo inclui componentes assíncronos em relação ao núcleo. Por esse motivo, o texto não transfere a observação de três ciclos para o ARM, o MMIO ou a rede.

A comparação de 60 ns com 315 ns produz o fator de 5,253 para os núcleos isolados. Essa razão compara duas implementações do algoritmo sob o procedimento descrito, não FPGA e software como plataformas completas. Quando a comunicação é incluída, o RTT médio de 10.228 ns é cerca de 32,4 vezes o tempo médio do núcleo C++; nessa configuração, o protótipo não reduz a latência da decisão quando usado pelo ARM dessa maneira.

6.3.2 Comunicação e vazão

Cada envio bem-sucedido exige, no mínimo, leitura dos dois ponteiros TX, escrita das oito palavras e publicação de `TX_HEAD`. A recepção exige leitura dos ponteiros RX, leitura das oito palavras e publicação de `RX_TAIL`. Consultas adicionais ocorrem enquanto não há espaço ou resposta. A diferença entre 60 ns internos e 10.228 ns de ida e volta é compatível com o custo acumulado de MMIO, espera ativa e execução no ARM, mas a campanha não decompõe esse custo por componente.

O máximo de 1.146.360 ns mostra que o caminho completo teve uma cauda muito maior que a média. Sem rastreamento do sistema operacional e do barramento, não é possível atribuir essa amostra exclusivamente ao escalonador Linux ou exclusivamente ao MMIO. A conclusão sustentada é mais restrita: a variabilidade aparece fora do contador do núcleo lógico.

A vazão de 102.381 mensagens/s corresponde a aproximadamente 6,55 MB/s de carga útil somando 32 bytes de evento e 32 bytes de resposta. Esse cálculo não representa todo o tráfego do barramento, pois exclui ponteiros e consultas repetidas. O valor superou o limiar de engenharia de 100.000 mensagens/s adotado pelos autores; ainda assim, o limiar não veio de uma exigência de bolsa e não permite afirmar adequação a HFT operacional.

6.3.3 Resultado do experimento de cálculo energético

A Tabela 6 aplica a Equação 7 às potências e aos tempos declarados na metodologia. O resultado é estimado, não medido. Em particular, a linha x86 combina duas hipóteses e não corresponde a uma execução realizada neste trabalho.

Tabela 6 – Estimativa de energia por decisão sob as hipóteses adotadas

Substrato	Potência adotada	Tempo adotado	Energia estimada
Lógica FPGA Cyclone V	50 mW	60 ns, registrado	3 nJ
Núcleo ARM Cortex-A9	1,5 W	315 ns, registrado	0,47 μ J
Núcleo x86 ilustrativo	15 W	30 ns, hipótese	0,45 μ J

Fonte: Experimento de cálculo dos autores; potências e tempo x86 adotados como hipóteses.

Sob essas hipóteses, o valor calculado para a lógica FPGA é inferior aos valores calculados para os núcleos de processador. A diferença não deve ser interpretada como medição de eficiência da plataforma, porque os limites de potência não são equivalentes e o cálculo exclui ARM, memória, ponte MMIO, alimentação da placa e rede. O resultado indica apenas uma hipótese a ser verificada por instrumentação direta.

6.4 Discussão frente aos trabalhos relacionados

A Tabela 7 contextualiza os números sem tratá-los como ranking. As referências de FPGA incluem rede ou medem subsistemas com fronteiras diferentes; o protótipo mede ida e volta por uma ponte MMIO controlada pelo ARM. Frequência, dispositivo, protocolo e instrumentação também diferem.

Tabela 7 – Contextualização das latências reportadas

Referência	Valor reportado	Fronteira principal
Este trabalho	60 ns internos; 10.228 ns de RTT médio	Núcleo Cyclone V e ida e volta ARM-MMIO-FPGA.
Leber, Geib e Litz (2011)	inferior a 1 μ s	Processamento FPGA integrado ao caminho de rede descrito pelos autores.
Boutros <i>et al.</i> (2017)	269 ns no subsistema; 869 ns de ida e volta	Kintex UltraScale a 156 MHz com HLS e interface de rede.
Lockwood <i>et al.</i> (2012)	inferior a 1 μ s	Biblioteca FPGA com funções de rede e negociação.

Fonte: Elaborado pelos autores com dados de Leber, Geib e Litz (2011), Boutros *et al.* (2017) e Lockwood *et al.* (2012).

Os valores da literatura não isolam o efeito de custo, ferramenta HLS ou dispositivo. A comparação permite observar apenas que retirar o processador de propósito geral do caminho

crítico é uma decisão arquitetural recorrente nos trabalhos de menor latência. Implementar rede diretamente no FPGA seria outro projeto e permanece fora do escopo desta monografia.

6.4.1 Limitações da evidência

Os testes demonstram os comportamentos codificados e o ensaio de placa demonstra integração no cenário registrado. A campanha fornece uma caracterização inicial, mas carece de repetições independentes, arquivo bruto versionado, medição de uso de CPU e decomposição do RTT. O fluxo de 5 mensagens/s não foi usado para determinar a vazão; o programa de medição usa eventos pré-construídos. Também não houve medição elétrica. A energia por decisão foi apenas calculada com potências de referência e limites físicos diferentes; por isso, não se conclui eficiência energética da plataforma nem se extrapolam os valores para uma instalação de mercado.

7 Conclusão

Este trabalho desenvolveu um protótipo acadêmico que recebe mensagens FAST sintéticas em software, converte seus campos para um contrato de oito palavras, atualiza no FPGA um livro de ofertas por nível e aplica uma regra gerada a partir de MATLAB. A regra foi mantida separada da ponte MMIO e do livro, conforme o objetivo de estabelecer uma infraestrutura fixa ao redor de um bloco de decisão substituível.

Os objetivos funcionais foram atendidos no escopo definido. O contrato ARM-FPGA foi documentado e exercitado por testes C++ e VHDL; o livro mantém oito níveis por lado; a estratégia produz NOOP, BUY e SELL; e um registro de execução na DE10-Nano mostra eventos do fluxo sintético correlacionados a respostas do hardware. Esses resultados validam a integração implementada, mas não validam negociação em mercado, pois o sistema não recebe dados de bolsa, não envia ordens e não possui posição, risco ou conformidade.

A campanha quantitativa registrada separou o núcleo lógico da comunicação. O contador do FPGA indicou três ciclos, ou 60 ns a 50 MHz, e não resolveu diferença entre mínimo e máximo na campanha de um milhão de eventos; nessa resolução de 20 ns, a variação interna foi considerada desprezível. A implementação C++ de referência apresentou média de 315 ns por evento, razão de 5,253 em relação ao núcleo FPGA. Ao incluir o ARM, a consulta das filas e o MMIO, a latência média de ida e volta foi 10.228 ns, o percentil 99 foi 10.310 ns e o máximo chegou a 1.146.360 ns. A vazão em fluxo foi 102.381 mensagens/s.

Esses valores sustentam duas conclusões restritas. O núcleo FPGA executou o conjunto testado em uma contagem fixa de ciclos e em menos tempo que o núcleo C++ isolado. No caminho completo, porém, essa vantagem não se manteve: na configuração adotada, a ida e volta foi cerca de 32,4 vezes o tempo médio do núcleo C++. A campanha não decompôs o custo entre MMIO, espera ativa, barramento e sistema operacional; por isso, não se atribui causalidade exclusiva a um desses componentes.

As principais limitações são o fluxo sintético de baixa taxa, o livro agregado e de profundidade fixa, a regra não calibrada, a ausência de ordens e risco e a existência de apenas uma campanha quantitativa preservada no texto. A saída bruta da campanha e repetições independentes não foram versionadas, o que impede calcular incerteza entre execuções. A energia por decisão foi estimada pelo produto entre potências de referência e tempos adotados, sem medição elétrica e com limites físicos distintos; o cálculo não sustenta, por si só, uma conclusão sobre a eficiência energética da plataforma completa.

Como continuidade, propõem-se ações ligadas diretamente a essas limitações:

- versionar a saída bruta de cada campanha e repetir medições após reinicializações, registrando mediana, p90, p95, p99, máximo e condições de execução;
- instrumentar separadamente escrita MMIO, espera da resposta e leitura MMIO para localizar o custo do caminho completo;

- medir a potência na placa durante uma carga controlada para substituir as hipóteses do cálculo por energia por evento obtida experimentalmente;
- substituir o gerador simples por repetição de dados históricos autorizados e por cenários de rajada, mantendo explícita a diferença para um fluxo de bolsa ao vivo;
- ampliar o modelo de eventos para cancelamentos e execuções parciais e verificar o efeito sobre área, frequência e latência;
- adicionar posição e limites de risco antes de estudar qualquer canal de emissão de ordens;
- avaliar outras regras MATLAB e registrar como cada regeneração altera recursos, temporização e comportamento funcional.

Conclui-se que o artefato constitui uma base acadêmica para estudar a integração ARM–FPGA aplicada a um fluxo simplificado de dados de mercado e à geração de uma regra por HLS. Sua contribuição está nos contratos, na separação entre estado e decisão e nas evidências funcionais e quantitativas iniciais, não na reprodução de uma plataforma de HFT operacional.

Referências

- ALDRIDGE, I. **High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems**. 2. ed. [S.l.]: Wiley, 2013.
- BOUTROS, A. *et al.* Build fast, trade fast: Fpga-based high-frequency trading using high-level synthesis. *In: 2017 International Conference on ReConFigurable Computing and FPGAs*. Cancun, Mexico: IEEE, 2017. p. 1–6.
- BROGAARD, J.; HENDERSHOTT, T.; RIORDAN, R. High-frequency trading and price discovery. **The Review of Financial Studies**, v. 27, n. 8, p. 2267–2306, 2014.
- CONT, R.; STOIKOV, S.; TALREJA, R. A stochastic model for order book dynamics. **Operations Research**, v. 58, n. 3, p. 549–563, 2010.
- CORMEN, T. H. *et al.* **Introduction to Algorithms**. 4. ed. [S.l.]: MIT Press, 2022.
- EDDY, W. M. **Transmission Control Protocol**. [S.l.], 2022. Disponível em: <https://www.rfc-editor.org/rfc/rfc9293>. Acesso em: 6 jul. 2026.
- FIX Trading Community. **FIX Adapted for STreaming Technical Standard**. [S.l.], 2006. Disponível em: <https://www.fixtrading.org/standards/fast-online/>. Acesso em: 13 abr. 2026.
- GOULD, M. D. *et al.* Limit order books. **Quantitative Finance**, v. 13, n. 11, p. 1709–1742, 2013.
- HARRIS, D. M.; HARRIS, S. L. **Digital Design and Computer Architecture**. 2. ed. [S.l.]: Morgan Kaufmann, 2013.
- HASBROUCK, J.; SAAR, G. Low-latency trading. **Journal of Financial Markets**, v. 16, n. 4, p. 646–679, 2013.
- HENDERSHOTT, T.; JONES, C. M.; MENKVELD, A. J. Does algorithmic trading improve liquidity? **The Journal of Finance**, v. 66, n. 1, p. 1–33, 2011.
- Intel Corporation. **Embedded Peripherals IP User Guide: DMA Controller**. [S.l.], 2019. Document UG-01085.
- Intel Corporation. **Cyclone V and Arria V SoC Device Design Guidelines (AN 796)**. [S.l.], 2022. Disponível em: <https://www.intel.com/content/www/us/en/docs/programmable/683360/18-0/lightweight-hps-to-fpga-bridge.html>. Acesso em: 12 maio 2026.
- Intel Corporation. **Avalon Interface Specifications**. [S.l.], 2024. Disponível em: <https://www.intel.com/content/www/us/en/docs/programmable/683091/current/avalon-memory-mapped-interfaces.html>. Acesso em: 13 abr. 2026.
- Intel Corporation. **Intel Quartus Prime Platform Designer User Guide**. [S.l.], 2024. Disponível em: <https://www.intel.com/content/www/us/en/docs/programmable/683609/current/platform-designer-system-integration-tool.html>. Acesso em: 13 abr. 2026.
- Intel Corporation. **Cyclone V FPGA and SoC FPGA Product Brief**. [S.l.], 2025. Disponível em: <https://cdrdv2-public.intel.com/853455/cyclone-v-fpgas-and-socs-product-brief.pdf>. Acesso em: 13 abr. 2026.
- JAIN, R. **The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling**. [S.l.]: John Wiley & Sons, 1991.

JEONG, E. *et al.* mTCP: A highly scalable user-level TCP stack for multicore systems.

In: Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI). [S.l.]: USENIX Association, 2014. p. 489–502.

LEBER, C.; GEIB, B.; LITZ, H. High frequency trading acceleration using fpgas. *In: 2011 21st International Conference on Field Programmable Logic and Applications.* [S.l.: s.n.], 2011. p. 317–322.

LOCKWOOD, J. W. *et al.* A low-latency library in fpga hardware for high-frequency trading. *In: 2012 IEEE 20th Annual Symposium on High-Performance Interconnects.* [S.l.: s.n.], 2012. p. 9–16.

Object Computing, Inc. **mFAST: a C++ FIX/FAST Encoder and Decoder.** [S.l.], 2026.

Disponível em: <https://github.com/objectcomputing/mFAST>. Acesso em: 13 abr. 2026.

PATTERSON, D. A.; HENNESSY, J. L. **Computer Organization and Design: The Hardware/Software Interface.** 5. ed. [S.l.]: Morgan Kaufmann, 2017.

RIZZO, L. netmap: A novel framework for fast packet i/o. *In: Proceedings of the USENIX Annual Technical Conference (ATC).* [S.l.]: USENIX Association, 2012. p. 101–112.

Terasic Technologies. **DE10-Nano User Manual.** [S.l.], 2024. Disponível em: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=167&No=1046>.

Acesso em: 13 abr. 2026.

The MathWorks, Inc. **Fixed-Point Designer Documentation.** [S.l.], 2026. Disponível em:

<https://www.mathworks.com/help/fixedpoint/>. Acesso em: 13 abr. 2026.

The MathWorks, Inc. **HDL Coder Documentation.** [S.l.], 2026. Disponível em: <https://www.mathworks.com/help/hdlcoder/>.

Acesso em: 13 abr. 2026.

APÊNDICE A – Contrato de Comunicação ARM–FPGA

Este apêndice consolida o contrato de comunicação utilizado pelo protótipo. Ele complementa a descrição do Capítulo 4 e deve permanecer alinhado aos arquivos `cpp/src/fpga_shared_stream.h`, `cpp/src/fast_receiver.cpp`, `vhdl/arm_fpga_shared_stream_bridge.vhd` e `docs/arm-fpga-shared-memory-stream.md`.

A.1 Registradores

A Tabela 8 consolida os registradores de identificação, controle das filas e telemetria. Os campos de desempenho são produzidos pelo motor e lidos pelo ARM; escrever 1 em `PERF_CTRL` reinicia seus contadores.

Tabela 8 – Registradores da interface MMIO

Offset	Nome	Acesso	Dono	Descrição
0x000	MAGIC	RO	FPGA	Valor 0x48465431.
0x004	VERSION	RO	FPGA	Versão do protocolo.
0x008	CTRL	RW	ARM	Bit 0 reinicia ponteiros.
0x00C	STATUS	RO	FPGA	Bits de disponibilidade das filas.
0x010	TX_HEAD	RW	ARM	Publicação de comandos.
0x014	TX_TAIL	RO	FPGA	Consumo de comandos.
0x018	RX_HEAD	RO	FPGA	Publicação de respostas.
0x01C	RX_TAIL	RW	ARM	Consumo de respostas.
0x020	TX_DEPTH	RO	FPGA	Profundidade da fila TX.
0x024	RX_DEPTH	RO	FPGA	Profundidade da fila RX.
0x028	SLOT_WORDS	RO	FPGA	Palavras por quadro.
0x030	PERF_CTRL	RW	ARM	Bit 0 reinicia contadores.
0x034	PERF_CLOCK_HZ	RO	FPGA	Frequência usada na conversão de ciclos.
0x038	PERF_COUNT	RO	FPGA	Quantidade de eventos medidos.
0x03C	PERF_LAST_LATENCY	RO	FPGA	Latência mais recente, em ciclos.
0x040	PERF_MIN_LATENCY	RO	FPGA	Menor latência, em ciclos.
0x044	PERF_MAX_LATENCY	RO	FPGA	Maior latência, em ciclos.
0x048-0x04C	PERF_SUM_LATENCY	RO	FPGA	Soma de latências em 64 bits.
0x050	PERF_CMD_STALL	RO	FPGA	Ciclos de espera no comando.
0x054	PERF_RSP_STALL	RO	FPGA	Ciclos de espera na resposta.

Fonte: Elaborado pelos autores conforme `fpga_shared_stream.h` e `arm_fpga_shared_stream_bridge.vhd`.

A.2 Regras de filas

As filas TX e RX usam a mesma semântica:

- vazia: `head == tail;`
- cheia: `next(head) == tail;`

- capacidade útil: $DEPTH - 1$;
- produtor escreve payload antes de publicar `head`;
- consumidor lê payload antes de publicar `tail`.

Para $DEPTH = 64$ e $SLOT_WORDS = 8$, a base TX é 0×100 , cada slot tem 32 bytes e a base RX é 0×900 .

A.3 Formato de evento

O evento enviado do ARM para o FPGA usa oito palavras. A palavra de sequência é preservada para correlação entre entrada e saída. O identificador de símbolo é numérico para evitar processamento textual em hardware. O preço é escalado por 10^4 para representar casas decimais com inteiros.

A Tabela 9 apresenta a ordem das palavras. A ordem é parte do protocolo: alterar uma posição exige atualizar software, RTL, testes e documentação.

Tabela 9 – Formato do quadro ARM–FPGA

Palavra	Campo	Semântica
<code>word0</code>	Sequência	Número de sequência da mensagem sintética.
<code>word1</code>	Símbolo	Identificador numérico atribuído pelo receptor.
<code>word2</code>	Preço	Preço em escala 10^4 .
<code>word3</code>	Quantidade	Quantidade agregada no nível de preço.
<code>word4</code>	Tipo de evento	Atualização/inserção, remoção ou reset.
<code>word5</code>	Lado	Compra ou venda.
<code>word6</code>	Reserva	Campo reservado para extensão futura.
<code>word7</code>	Reserva	Campo reservado para extensão futura.

Fonte: Elaborado pelos autores conforme o contrato implementado em C++ e VHDL.

A.4 Formato de resposta

O software interpreta a resposta do FPGA como:

- `word0`: sequência;
- `word1`: ação;
- `word2`: melhor compra em escala 10^4 ;
- `word3`: quantidade na melhor compra;
- `word4`: melhor venda em escala 10^4 ;

- `word5`: quantidade na melhor venda;
- `word6`: *spread*;
- `word7`: desbalanceamento assinado.

A.5 Checklist de alteração do contrato

Quando o contrato ARM–FPGA for alterado, os seguintes itens devem ser revisados em conjunto:

- mapa de registradores no VHDL e na documentação;
- largura `G_SLOT_WORDS` no VHDL e `kFrameWords` no C++;
- cálculo da base RX no software, nos bancos de teste e no texto;
- ordem das palavras em `fast_receiver.cpp`;
- decodificação das palavras em `order_book_core.vhd`;
- montagem da resposta em `generated_strategy_core.vhd`;
- testes C++ e VHDL que validam fila cheia, *wrap-around* e resposta;
- comandos e exemplos de execução no README e neste apêndice.

Esse checklist existe porque mudanças pequenas em interfaces binárias podem causar erros difíceis de diagnosticar. Em particular, uma alteração da largura da posição muda a base RX; uma alteração da ordem das palavras pode fazer preço ser interpretado como quantidade; e uma alteração de evento sem atualização do livro pode gerar estados de topo inconsistentes.

APÊNDICE B – Comandos de Construção, Teste e Execução

Este apêndice reúne comandos úteis para reproduzir a construção e validação do protótipo.

B.1 Validação local

A Listagem 3 reúne os alvos de validação local usados no trabalho.

Listagem 3 – Comandos de teste local

```
1 make docker-image
2 make cpp-test
3 make cpp-smoke
4 make vhdl-test-all
```

Fonte: Elaborado pelos autores.

O alvo `cpp-test` valida o invólucro MMIO com arquivo temporário. O alvo `cpp-smoke` executa o gerador e o receptor em modo software. O alvo `vhdl-test-all` executa os bancos de teste da ponte, do livro, da estratégia, do motor completo e do invólucro Avalon-MM.

B.2 Geração da estratégia

A Listagem 4 mostra os comandos de preparação do ambiente MATLAB e geração da estratégia.

Listagem 4 – Geração do bloco de estratégia por HDL Coder

```
1 make matlab-docker-image
2 make matlab-hdl-generate
```

Fonte: Elaborado pelos autores.

O primeiro comando prepara a imagem esperada para execução do MATLAB em ambiente controlado. O segundo executa a rotina `matlab/hdl_generator.m`, que chama a geração VHDL da função `strategy.m`. Após essa etapa, recomenda-se executar os bancos de teste da estratégia e do motor completo, pois uma mudança no bloco gerado pode alterar latência, portas ou comportamento.

B.3 Geração HDL e compilação para DE10-Nano

A Listagem 5 apresenta a sequência de preparação, compilação, programação e implantação na placa.

Após a programação do FPGA, o gerador e o receptor podem ser executados na placa com os comandos da Listagem 6:

Listagem 5 – Fluxo previsto para DE10-Nano

```
1 make de10-sysroot
2 make matlab-login
3 make build
4 make quartus-program
5 make de10-enable-bridges
6 make deploy
```

Fonte: Elaborado pelos autores.

Listagem 6 – Execução na placa

```
1 ssh root@192.168.7.1 'cd /home/root && ./fast_data_feed '
2 ssh root@192.168.7.1 \
3 'cd /home/root && HFT_FPGA_MMIO_BASE=0xFF200000 ./fast_receiver '
```

Fonte: Elaborado pelos autores.

O endereço 0xFF200000 corresponde à base típica da ponte leve HPS-FPGA na DE10-Nano quando o componente está no offset zero do Platform Designer. Se o componente for posicionado em outro offset, a base deve ser ajustada.

B.4 Variáveis de ambiente

A Tabela 10 relaciona as variáveis que habilitam e configuram o acesso físico. Na ausência da variável de base, o receptor não tenta mapear o FPGA.

Tabela 10 – Variáveis de ambiente do receptor

Variável	Função
HFT_FPGA_MMIO_BASE	Base física do periférico. Quando ausente, o receptor opera somente em software.
HFT_FPGA_MMIO_SPAN	Tamanho da região mapeada. Quando ausente, usa valor padrão suficiente para a geometria esperada.
HFT_FPGA_MMIO_DEV	Dispositivo usado para mapeamento MMIO. Por padrão, /dev/mem.

Fonte: Elaborado pelos autores conforme `fast_receiver.cpp`.

A presença de `HFT_FPGA_MMIO_BASE` muda o modo de operação do receptor. Por isso, testes de software devem ser executados sem essa variável, enquanto testes em placa devem defini-la explicitamente. Em ambientes compartilhados, recomenda-se imprimir a base efetiva antes de enviar eventos para evitar confusão entre offset de Platform Designer e base da ponte leve HPS–FPGA.

B.5 Sequência recomendada de depuração

Uma sequência prática de depuração é:

1. executar `make cpp-test` para validar a semântica de filas no invólucro;
2. executar `make cpp-smoke` para validar FAST, TCP e decodificação sem FPGA;
3. executar `make vhdl-test-all` para validar os blocos de hardware em simulação;
4. gerar e programar o arquivo de configuração da FPGA;
5. habilitar pontes HPS–FPGA;
6. executar o receptor com `HFT_FPGA_MMIO_BASE` e verificar leitura de `MAGIC`;
7. iniciar o gerador e observar respostas `[FPGA->ARM]`.

Se a leitura de `MAGIC` falhar, o problema provavelmente está em endereço, arquivo de configuração ou ponte HPS–FPGA. Se `MAGIC` estiver correto, mas `Send` falhar por fila cheia, deve-se investigar o consumo da fila TX, a reinicialização e os sinais internos de validade e prontidão. Se respostas não aparecerem, mas TX for consumida, o foco passa a ser o livro de ofertas, a estratégia e a fila RX.